

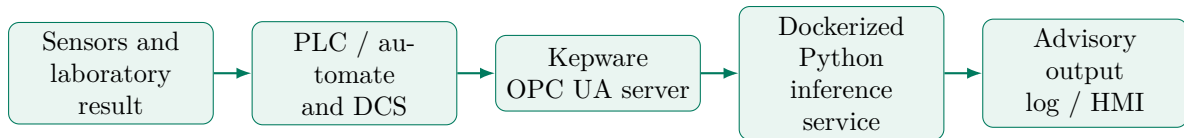
Industrial Deployment Prototype for SLURRY_FREE_ACID Estimation

Ikram Benfella

1 September 2026

Purpose. I prepared the first isolated deployment prototype that connects the selected Python model to an OPC UA data source. The current system is a laboratory test using CSV replay.

1 Architecture and responsibility boundary



Component	Role	Safety boundary
Sensors / actuators	Measure the process; valves, pumps and motors act on it.	The ML service does not directly access actuator commands.
PLC / DCS	Deterministic control, interlocks, alarms and operator supervision.	Plant safety remains independent of model availability.
Kepware	Translates controller-specific protocols and exposes approved tags through OPC UA.	It is a communication gateway, not a prediction model.
Docker service	Validates measurements, keeps recent history, builds model features and runs inference.	Advisory only; no setpoint, interlock, valve or pump control.
Operator / HMI	Displays and interprets the advisory result.	The operator remains the decision-maker.

2 What I implemented

The work is organized under `deployment/` so that deployment logic is separated from the research notebooks.

Element	Implementation	Purpose
OPC UA simulator	A separate Python container replays the historical one-minute CSV and exposes configured OPC nodes.	Test the client/server boundary without connecting to the plant.
Inference service	A second Python container subscribes to a commit-sequence tag, reads coherent snapshots and loads the saved model artifacts.	Reproduce the intended live inference flow.
Docker Compose	Creates both containers on a private Docker network; the simulator exposes port 4840.	Reproducible and isolated Stage 1 environment on Windows/Docker Desktop.
Tag mapping	<code>config/opc_tags.yaml</code> maps research variable names to OPC Node IDs, units and required-tag metadata.	Avoid hard-coded plant addresses; real Node IDs can be substituted later.
Service policy	<code>config/service.yaml</code> centralizes mode, cadence, buffer, quality and output settings.	Keep operational choices visible and auditable.
Data-quality shield	Rejects missing required tags, bad/uncertain OPC quality, invalid chronology, excessive gaps and configured impossible values.	Prevent inference from unreliable inputs.
Runtime outputs	Health status, complete JSONL prediction history and one readable latest-prediction JSON file.	Monitoring, audit and simple human inspection.
PowerShell scripts	Start, stop and readable status commands.	Make the laboratory prototype easy to operate.

Model operating modes

- **Target-free mode (current default):** process measurements only $\rightarrow$ estimate of the current SLURRY_FREE_ACID. The deployed candidate is the selected five-cluster Ridge virtual sensor. This is a soft sensor, not a ten-minute forecast.
- **Target-anchored mode:** process measurements plus a sufficiently recent free-acid value $y_t \rightarrow$ predicted change $\widehat{\Delta y_{t,h}}$, reconstructed as

$$\widehat{y}_{t+h} = y_t + \widehat{\Delta y_{t,h}}.$$

Because free acid is currently a manual laboratory result, this mode requires a reliable laboratory-result timestamp and freshness policy.

3 Streaming, warm-up and output logic

1. The simulator updates all process tags, the source timestamp, then a sequence tag as the final commit marker.
2. The inference client subscribes to the sequence tag rather than polling continuously.
3. For each notification, the sequence is read before and after the process values. The snapshot is accepted only when both reads agree; this prevents mixing two CSV rows.
4. The data-quality shield validates the snapshot.
5. Accepted rows enter a chronological rolling buffer. The model then recreates the feature order and preprocessing used during training.

6. The prediction and its context are written to audit files. Advisory OPC writes remain disabled by default.

Why 31 rows are required. The feature pipeline has a 30-minute maximum lookback. It therefore needs 30 historical minutes plus the current row:

$$\text{minimum rows} = 30 + 1 = 31.$$

The service reports `WARMING_UP` rather than inventing unavailable history. Genuine missing required measurements can interrupt continuity and restart warm-up. In production, a 31-minute restart delay should be avoided by backfilling the last 30 minutes from the plant historian and, if approved, restoring a validated persistent local buffer. A separately trained raw-only fallback model is another option, but it would require its own validation.

Output	Meaning
<code>runtime/health.json</code>	Machine-readable state such as <code>STARTING</code> , <code>CONNECTED</code> , <code>WARMING_UP</code> , <code>RUNNING</code> , <code>DATA_ERROR</code> or <code>DISCONNECTED</code> , with an explicit reason.
<code>runtime/latest_prediction.json</code>	One formatted latest successful prediction for inspection.
<code>runtime/predictions.jsonl</code>	Append-only prediction history for audit and later comparison with laboratory results.
<code>scripts/status_stage1.ps1</code>	Short human-readable status: collected/required rows, reason for an error and latest prediction.

4 Stage 1 validation and corrections

The complete laboratory path was executed: Docker images were built, the OPC UA server started, the inference client connected and subscribed, model artifacts loaded, and prediction records were generated. The following engineering issues were identified and corrected during testing.

Observed issue	Cause	Correction
Docker Hub TLS timeout	Temporary registry/network timeout while downloading the base image.	Added explicit pull checking, retry logic and a clear network diagnostic; TLS verification was not disabled.
PowerShell Compose call failed	Docker arguments were not forwarded as an array.	Corrected the wrapper to pass all arguments and check exit codes.
LightGBM could not load	Linux image lacked the OpenMP runtime.	Installed <code>libgomp1</code> and added a build-time import check.
Client initially read the last replayed row	The simulator continued while the inference container had previously failed.	Restarted the complete laboratory environment from a clean replay state.
Duplicate/non-monotonic timestamps	Replay at 0.10 s was faster than the OPC publication/reading cycle.	Changed replay to 0.50 s, subscription to 100 ms, added coherent sequence-guarded reads and ignored already processed sequence values.
Prediction file absent during warm-up	It was created only after the first prediction.	Create the history file at startup and expose a separate readable latest-prediction file.
Unreadable protocol logs	OPC library INFO messages included full protocol packets.	Reduced library logging and added a concise status script.

Current interpretation of DATA_ERROR. A message reporting a missing required tag, for example FLOW_RATE_GROUND_PHOSPHATE, is a data-quality decision, not a container failure. The model refuses that instant because a required measurement is absent. The latest successful prediction may remain available while the current service status returns to warm-up.

5 Current limits and next steps

Already demonstrated

- isolated two-container OPC UA inference path;
- configuration-based tag mapping and model loading;
- subscription, reconnect, chronological buffering and warm-up behavior;
- explicit quality rejection and auditable prediction output;
- advisory-only safety boundary.

Still required before plant shadow mode

1. Confirm the DCS/PLC vendors, Kepware version, OPC UA endpoint, security policy, certificates and firewall route.
2. Replace placeholder Node IDs; confirm engineering units, physical limits and OPC quality semantics for every tag.
3. Add historian backfill/persistent-buffer recovery so a restart does not require 31 new live minutes.

Conclusion. Stage 1 proves that the research model can be packaged and connected through an OPC UA boundary in an isolated environment. It does not yet prove production accuracy or authorize operational intervention. The next technical milestone is a secured laboratory Kepware connection, followed by read-only shadow validation using correctly timestamped plant and laboratory data.